NEURAL NETWORKS

Milica Pavlović 2023/0148

Anastasija Ljubičić 2023/312

Group 9

Convolutional Neural Network Design

1. Problem Description

This project addresses the problem of automatic classification of images of concrete surfaces with the goal of detecting cracks. The task can be viewed as a binary image classification problem, where the model has to determine, based on the input image, whether or not there is a crack on the surface.

The input data are images of concrete surfaces of size 227 × 227 pixels with RGB channels. Each image belongs to one of two classes:

- **Positive** – the image contains a crack
- **Negative** – the image does not contain a crack

The dataset used in this project contains a total of **40000 images**, of which:

- **20000 images belong to the Positive class**
- **20000 images belong to the Negative class**

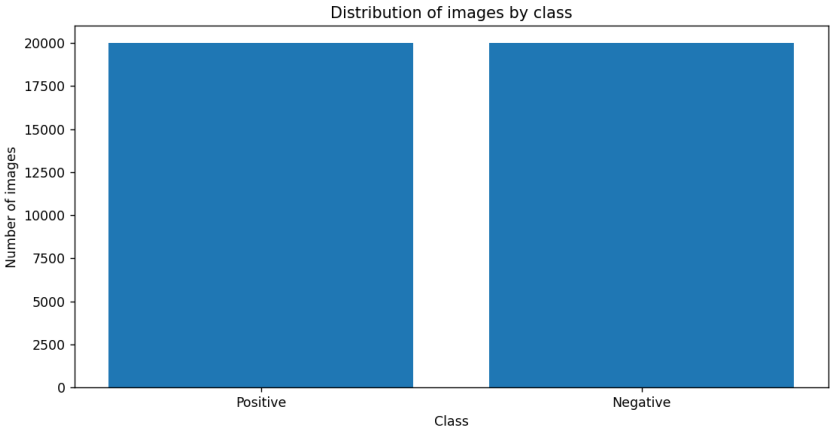
Based on these values, it can be concluded that the dataset is **balanced**, since both classes have the same number of samples.

Positive images: 20000

Negative images: 20000

Total images: 40000

The figure shows the distribution of samples per class. Both classes contain the same number of samples, which means that the dataset is balanced and no additional class-balancing techniques are needed.



2. Dataset Analysis

The dataset was split into training, validation and test sets.
70% of the data was used for training, while the remaining 30% was split equally between the validation and test sets.
The split was stratified, i.e. the class ratio was preserved in every subset.

Train size: 28000

Validation size: 6000

Test size: 6000



3. Data Preprocessing

In the preprocessing stage, all images were resized to a uniform size of 224 × 224 pixels (the original images are 227 × 227).
Pixel values were normalized to the range [0, 1] by dividing by 255; this normalization is applied to the training, validation and test sets alike.
Data augmentation techniques were applied to the training set, including slight rotation, zooming and horizontal flipping, with the aim of increasing model robustness and reducing overfitting. Augmentation is applied **only to the training set**, so the validation and test sets contain the original (rescaled) images.

Applied transformations (1 is applied to all sets, 2–4 only to the training set):

1. Rescaling (1/255) – preprocessing

- Normalization of pixel values to the range [0, 1]
- More stable and more efficient training

2. Horizontal Flip

- Horizontal mirroring of images
- Increases the diversity of the training data

3. Rotation Range (10°)

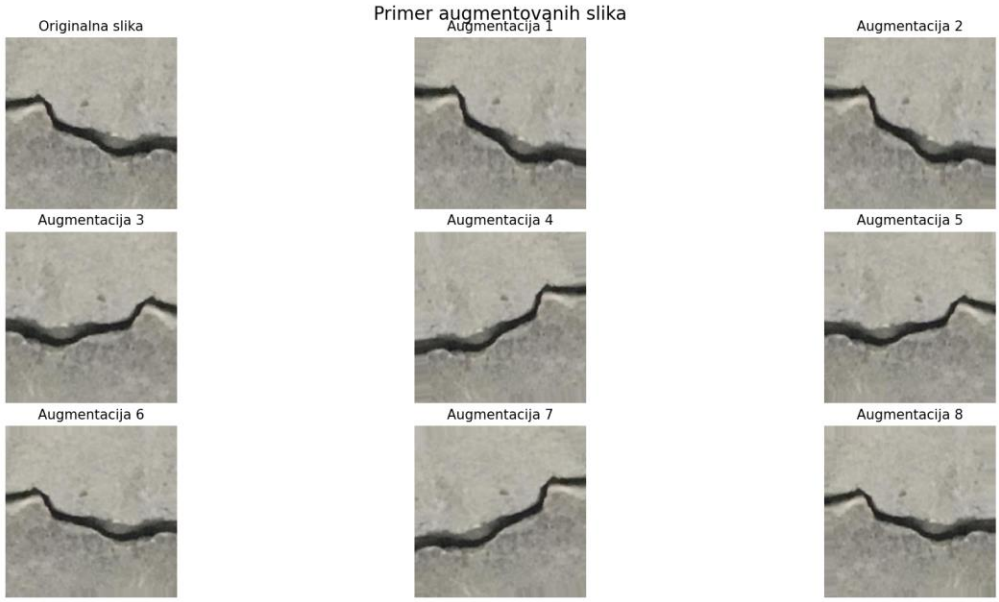
- Slight rotation of images
- Improves robustness of the model to small changes in crack orientation

4. Zoom Range (0.1)

- Slight zooming in and out
- The model becomes more robust to changes in scale and surface detail

Overall effect of augmentation:

- reduced overfitting
- better generalization of the model
- greater diversity of the training set without adding new real samples



The applied transformations allow the model to see different variations of the same sample during training, which increases robustness and reduces the risk of overfitting.

4. Convolutional Neural Network Architecture

A convolutional neural network with three convolutional blocks was designed for image classification.
Each block consists of a convolutional layer (3 × 3 kernels, 32, 64 and 128 filters respectively) with the ReLU activation function and a max-pooling layer that reduces dimensionality.
After feature extraction, a flatten layer and a fully connected layer with 128 neurons are used.
To reduce overfitting, a dropout layer with a rate of 0.5 is added.
The output of the network is a single neuron with the sigmoid activation function, since the task is binary classification. The network has 11,169,089 trainable parameters, about 99% of which belong to the fully connected layer after flattening.

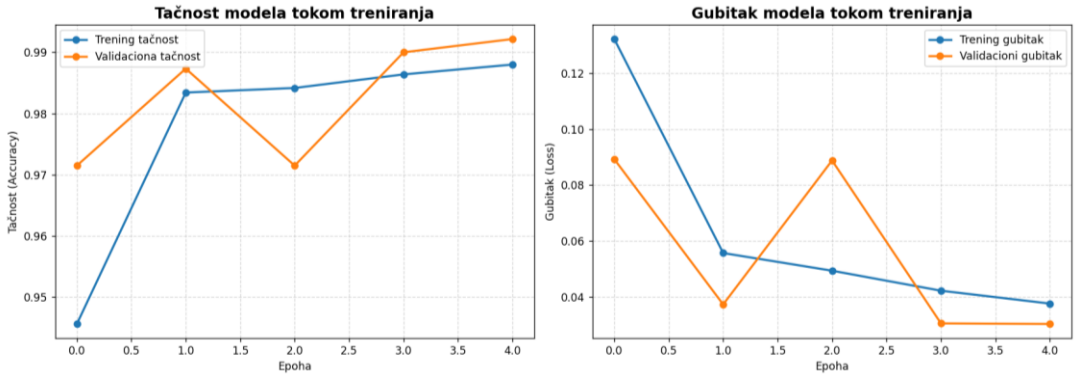
Loss Function and Optimization

Binary cross-entropy was used as the loss function, and the Adam optimizer was chosen for optimization.
The ReLU activation function was used in the hidden layers because of its efficiency and stability during training.

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 222, 222, 32)	896
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_1 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_2 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 128)	0
flatten (Flatten)	(None, 86528)	0
dense (Dense)	(None, 128)	11,075,712
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

Total params: 11,169,089 (42.61 MB)
Trainable params: 11,169,089 (42.61 MB)
Non-trainable params: 0 (0.00 B)

5. Performance Plots Across Epochs



During training, accuracy and loss were tracked across epochs (the model was trained for 5 epochs, numbered 0–4 on the plots).
Training accuracy increases and training loss decreases in every epoch. Validation accuracy and loss follow the same overall trend, with a temporary dip in epoch 2 (validation accuracy drops to about 97.2% and validation loss rises), after which both improve again. This indicates that the model learns successfully.

Validation accuracy stays close to training accuracy throughout training, which indicates that the model is not overfitting and generalizes well to new data. Validation accuracy is even slightly higher than training accuracy. This is expected: dropout and data augmentation are active only during training, and Keras reports training accuracy as an average over the whole epoch, while validation accuracy is measured with the model as it is at the end of the epoch.

At the end of training, the model achieved the following results:

- Final training accuracy: **98.80%**
- Final validation accuracy: **99.22%**
- Final training loss: **0.0376**
- Final validation loss: **0.0303**

These results show that the model distinguishes very successfully between images that contain cracks and images that do not.

6. Confusion Matrix

The confusion matrix shows the relationship between the true and the predicted classes.

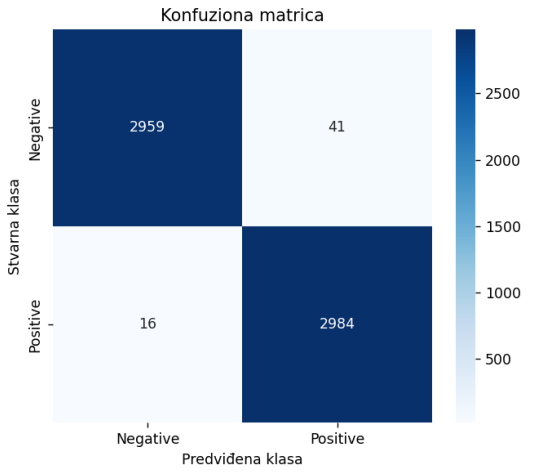
In our case the classes are:

- **Positive** – images that contain a crack
- **Negative** – images without a crack

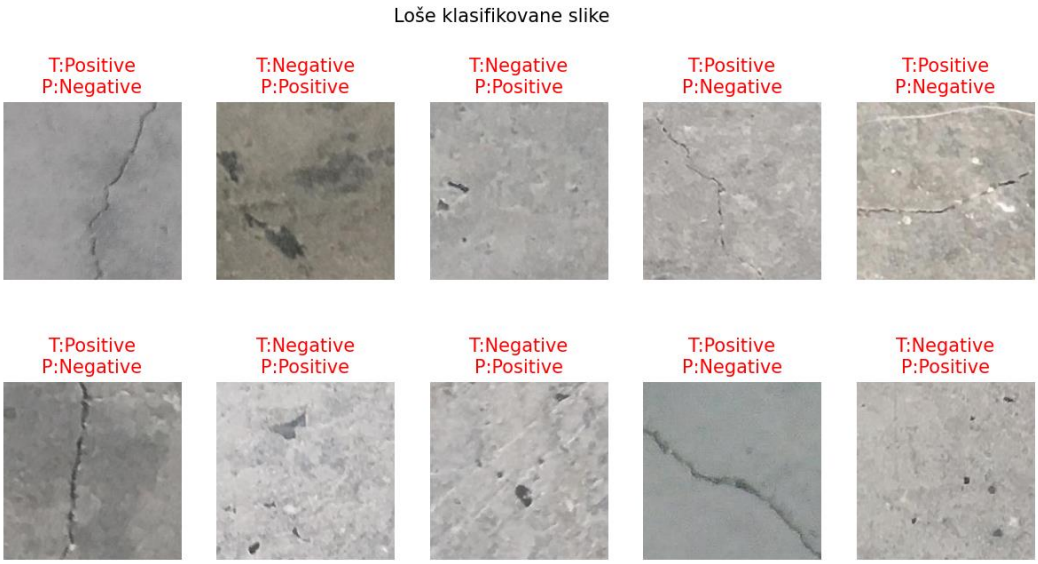
The results on the test set (6000 images) are as follows:

- **2959** images without a crack were correctly classified (true negatives)
- **2984** images with a crack were correctly classified (true positives)
- **41** images without a crack were incorrectly classified as cracked (false positives)
- **16** images with a crack were incorrectly classified as crack-free (false negatives)

In total, 57 of 6000 test images were misclassified, which corresponds to a test accuracy of 99.05%. For the Positive class (crack), precision is 98.64% and recall is 99.47%, giving an F1 score of 99.05%. The model therefore misses very few cracks, and most of its errors are false alarms on crack-free surfaces.



7. Misclassified and Correctly Classified Images



The figure shows examples of images that the model did not classify correctly.

In some images the cracks are very thin or barely visible, while in others the surface contains noise, stains or pores that the model mistakenly interprets as a crack.

These examples show the limitations of the model and point to situations in which additional data or a more complex architecture could improve the performance of the system. It should also be noted that the dataset was created by cropping patches from a limited number of high-resolution photographs, so patches from the same photograph may end up in both the training and the test set. The reported accuracy may therefore be slightly optimistic for images of completely new surfaces.



The figure shows examples of images that the model classified correctly.

The model successfully recognizes the presence of cracks on concrete surfaces, as well as images that do not contain cracks.

These results confirm that the convolutional neural network successfully learns the characteristic visual patterns that distinguish the two classes.

Fuzzy Controller Design

Controlled plant:

The controlled plant is the system we want to control, i.e. the system the controller acts on.

G(s) – transfer function of the plant. Its static gain is G(0) = 1/90 ≈ 0.011.

e^-4s – time delay of 4 seconds

$$G(s) = \frac{1}{s^4+26s^3+193s^2+510s+90}e^{-4s}$$

Reference range:

- the reference is the desired value of the output y(t)

$$r \in [-0.4, +0.4]$$

Control constraint:

- the control signal is the controller output and the plant input

- the output of the fuzzy controller must be designed so that it stays within the range [-100, 100]

$$u \in [-100, +100]$$

If u > 0 – the control action is positive

If u < 0 – the control action is negative

Because of the time delay and the complexity of the plant, the controller was not obtained by fuzzifying an existing classical controller; instead, it was tuned according to the desired behaviour of the system. For the same reasons, an **intuitive approach** was used for the controller design: the controller was built by observing and understanding the system, and the control action is adjusted according to the error – a large error requires a large control action. A Mamdani-type fuzzy system was used.

Controller structure

A fuzzy PI controller with 2 inputs and 1 output was used, in incremental form: the fuzzy system computes the change of the control signal, which is then integrated.

Input 1:

The error e(t) is the difference between the desired and the actual output. The current error tells how far the output is from the desired value and in which direction the controller should act. Given the reference range [-0.4, 0.4], the expected range of the error is [-0.8, 0.8].

Formula: e(t) = r(t) − y(t)

Input 2:

The change of error (Δe(t)) tells whether the error is increasing or decreasing and how fast the system is moving towards or away from the target. Its range is [-1, 1]. If Δe > 0, the value of the error is increasing, and if Δe < 0, it is decreasing. The magnitude of the error |e| decreases when e and Δe have opposite signs, i.e. when the output is approaching the reference. The larger |Δe| is, the faster the error changes.

Formula (discrete / continuous case): $\Delta e(k) = e(k) - e(k-1)$, or $de(t)/dt$

Output:

The output of the fuzzy controller is the change of the control signal $\Delta u(t)$. By accumulating it, the control signal $u(t)$ is obtained, which is the controller output and the plant input. The output shows how the controller wants to change the control action in order to improve the state of the system.

Formula: $u(k) = u(k-1) + \Delta u(k)$

- obtained by integrating the output of the fuzzy controller (in the continuous Simulink model, with an integrator block 1/s)

Operating principle of the controller:

- Large positive error – a large positive change of the control signal is needed
- Small positive error – a small positive change of the control signal is needed
- Large negative error – a large negative change of the control signal is needed
- Small negative error – a small negative change of the control signal is needed

Because Δu is integrated, the controller keeps adjusting the control signal $u(t)$ as long as the error is non-zero, so the error is driven towards zero (the integral action of the PI controller).

Fuzzy sets

Five fuzzy sets were used to describe the input and output variables of the controller:

• NB = negative big

• NS = negative small

• Z = zero

• PS = positive small

• PB = positive big

These sets allow numerical values to be represented linguistically, i.e. they determine to what degree a value belongs to a given fuzzy set. For every value of a variable, the membership function assigns a degree of membership from the interval [0, 1]. In this way, the inputs and outputs of the controller are not described sharply but gradually.

Steps of system operation:

1. the inputs E and dE are limited by saturation blocks and form the input vector of the controller
2. the fuzzy controller computes du
3. the integrator produces u
4. the saturation block limits u to [-100, 100] (the integrator itself is not limited, but since u never reaches the limit in the simulations, integrator windup does not occur)
5. u enters the plant G
6. the plant produces its output
7. the output passes through the time delay
8. the final output y is obtained
9. y is fed back through the feedback loop

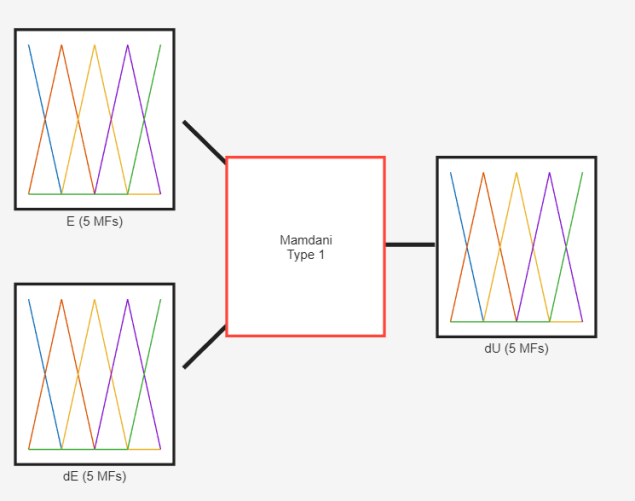


Figure 1 – Structure of the fuzzy inference system (FIS)

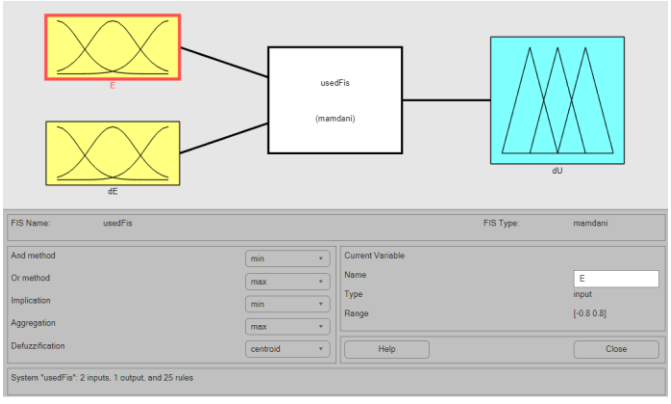


Figure 2 – FIS settings of the controller (Mamdani, 25 rules)

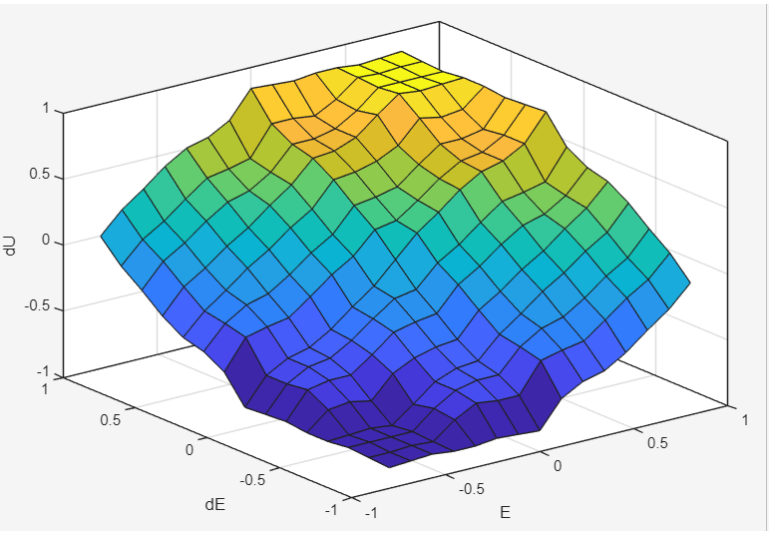


Figure 3 – Control surface

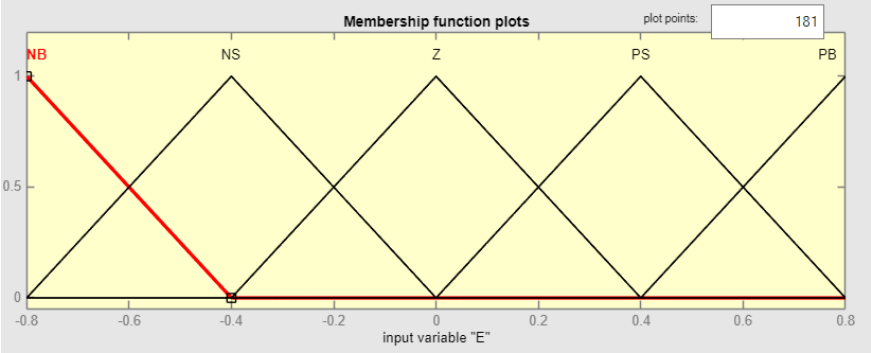


Figure 4 – Membership functions for E

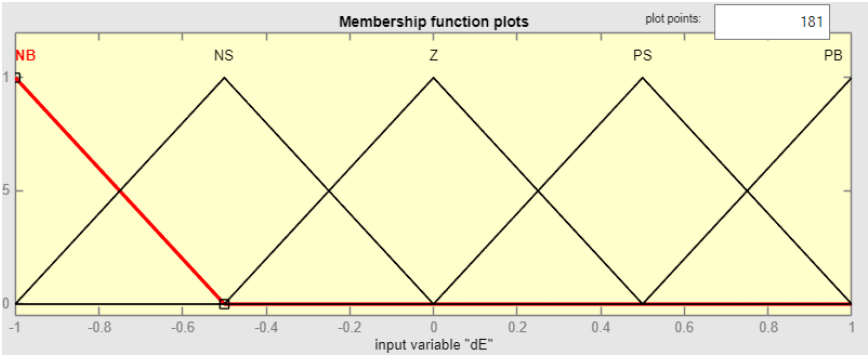


Figure 5 – Membership functions for dE

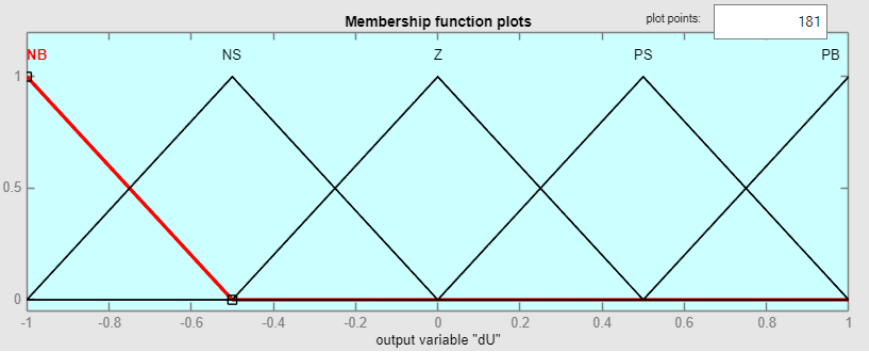


Figure 6 – Membership functions for the output dU

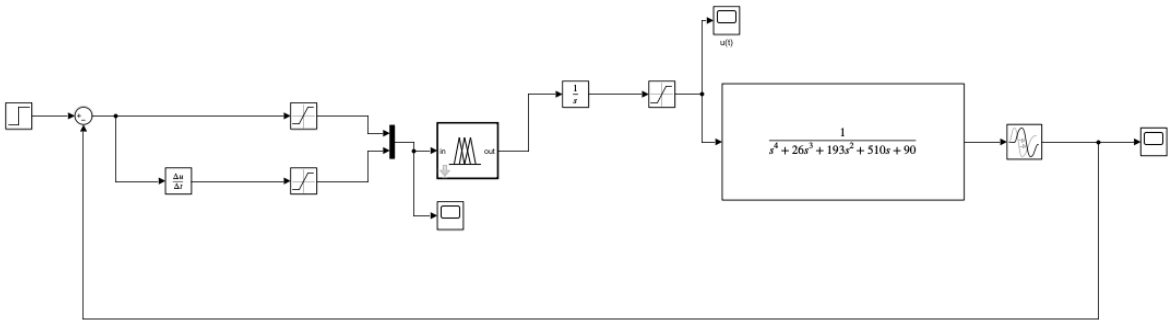


Figure 7 – Simulink model of system 1 (without disturbance)

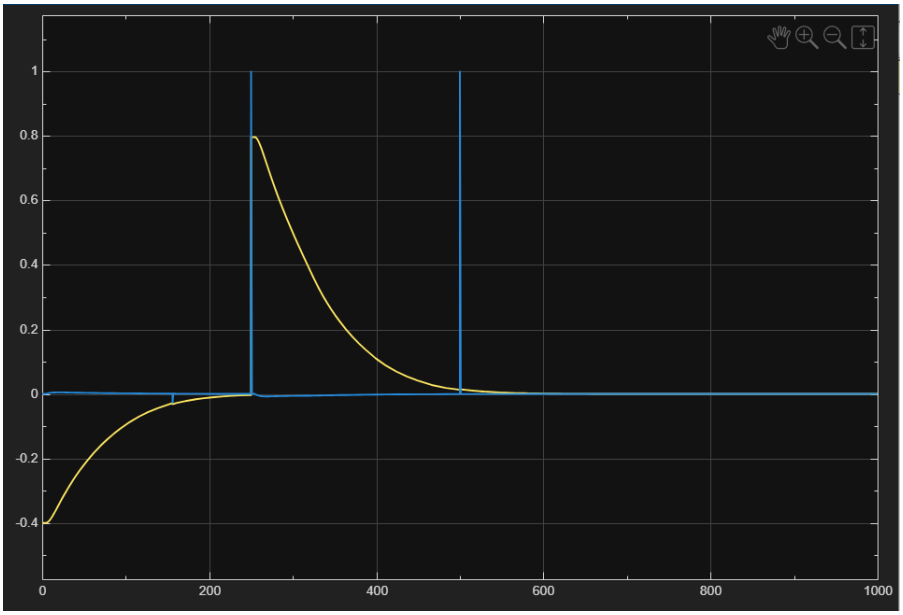


Figure 8 – Inputs of the fuzzy inference system

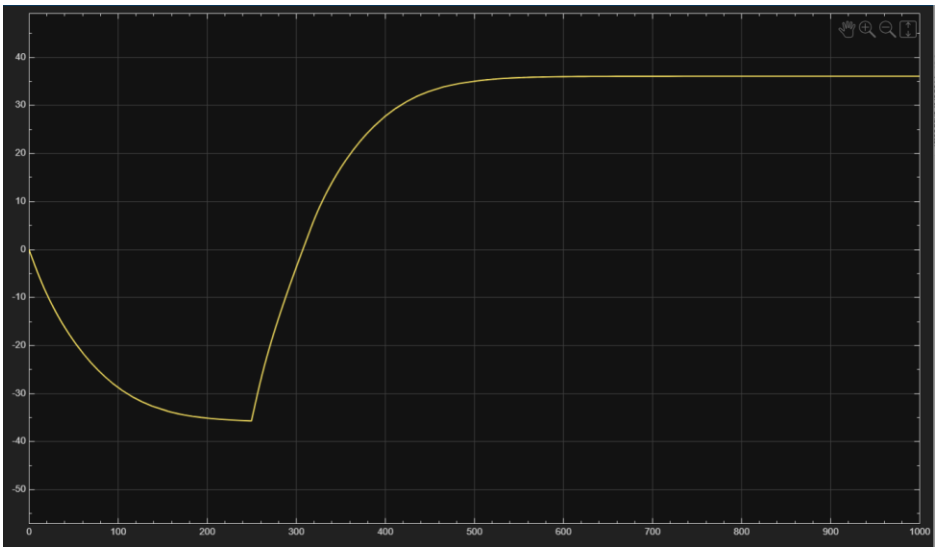


Figure 9 – Control signal

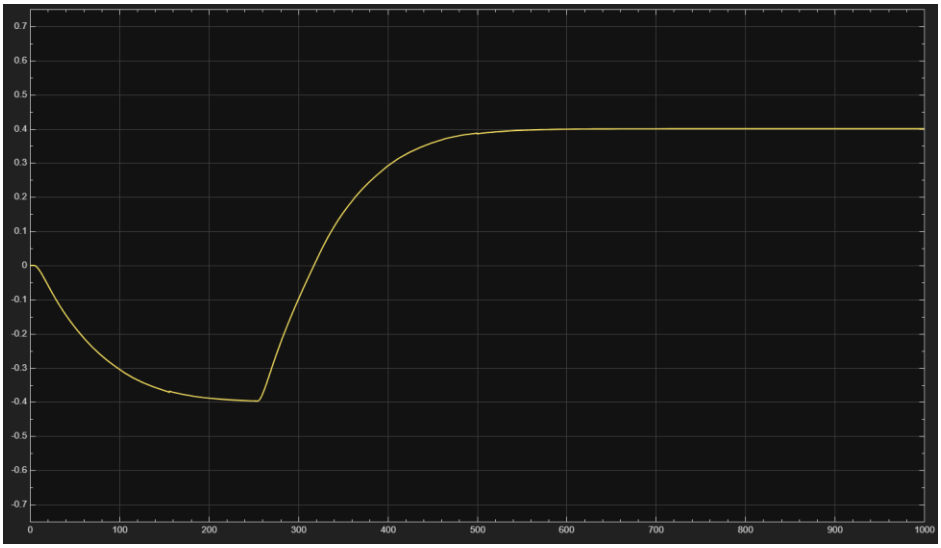


Figure 10 – Output signal

Discussion – system without disturbance:

Control signal

The control signal plot shows that at the beginning the controller generates a negative control signal in order to bring the system output to the minimum reference value. After the first steady state is reached, when the reference changes from the minimum to the maximum value, the control signal changes sign and moves continuously into the positive region. The control signal does not oscillate and has no abrupt jumps; it changes continuously and smoothly. Its amplitude

stays approximately within the interval from −36 to +36, which matches the static gain of the plant: to keep the output at y = ±0.4, a control signal of u = ±0.4 · 90 = ±36 is needed.

Plant output signal

Starting from its initial value, the output first approaches the minimum reference value −0.4 and then the maximum value +0.4. The response is smooth and stable, without oscillations or overshoot. It is relatively slow (settling takes roughly 250–300 s after the reference change), which is the price of the cautious controller tuning chosen because of the 4 s time delay and the plant dynamics.

Signals at the input of the fuzzy inference system

The inputs are the error E and the change of error dE. The plot shows that the error is largest at the moments when the reference changes.

After the reference changes from −0.4 to +0.4, the error instantly reaches approximately +0.8 and then decreases monotonically towards zero as the system output approaches the new reference value.

Conclusion:

Based on the obtained results, it can be concluded that the designed fuzzy PI controller provides stable and successful reference tracking in the case without disturbance.

System with disturbance (d)

A disturbance d is added to the plant input in addition to u, so the plant actually receives the input u + d. The controller tries to keep the system working properly, while the external influence d disturbs it.

The maximum disturbance amplitude was chosen as 50, so the disturbance is considered in the range from −50 to 50. This choice follows from the control constraint: in steady state the controller has to compensate the disturbance (u = −d) and at the same time keep the output at the reference, which in the worst case requires |u| = 36 + 50 = 86 < 100, so the control signal stays within the allowed range.

At the start of operation the disturbance is 0 (d = 0); at some moment it jumps to its maximum value of 50 (d = 50), and if the controller works correctly, the output then settles back around 0.

After that, the disturbance changes to −50 (d = −50).

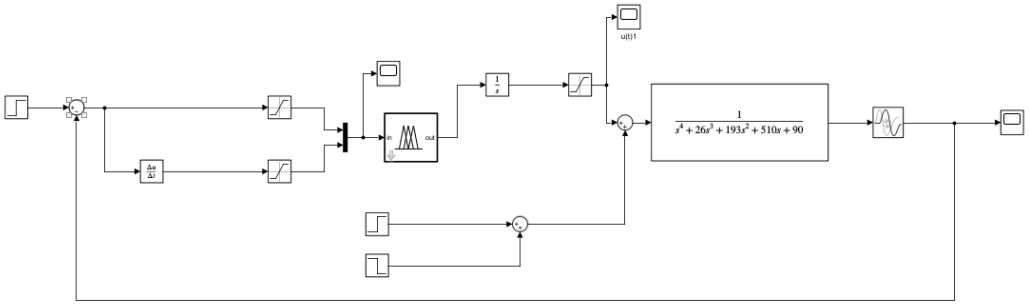


Figure 11 – Simulink model of system 2 (with disturbance)

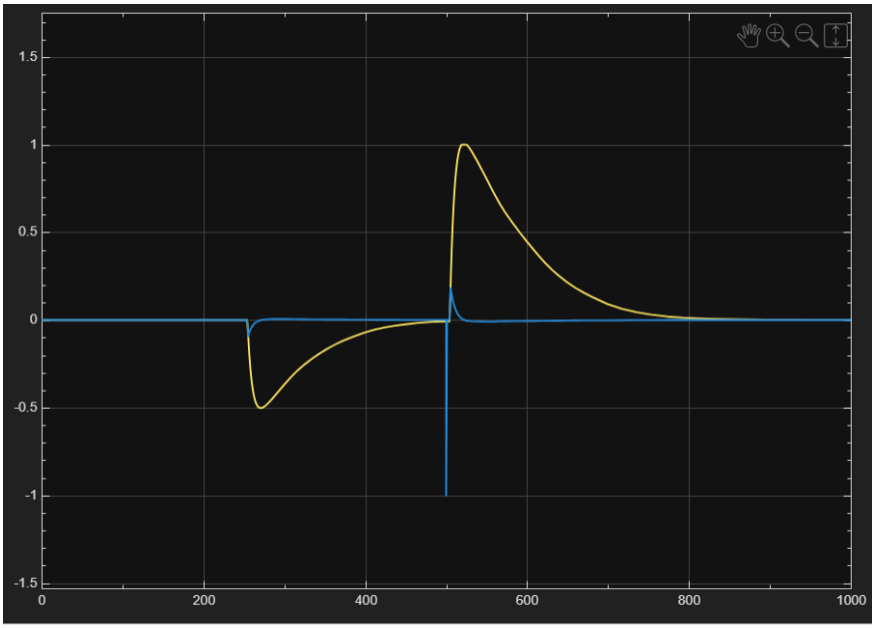


Figure 12 – Inputs of the fuzzy inference system

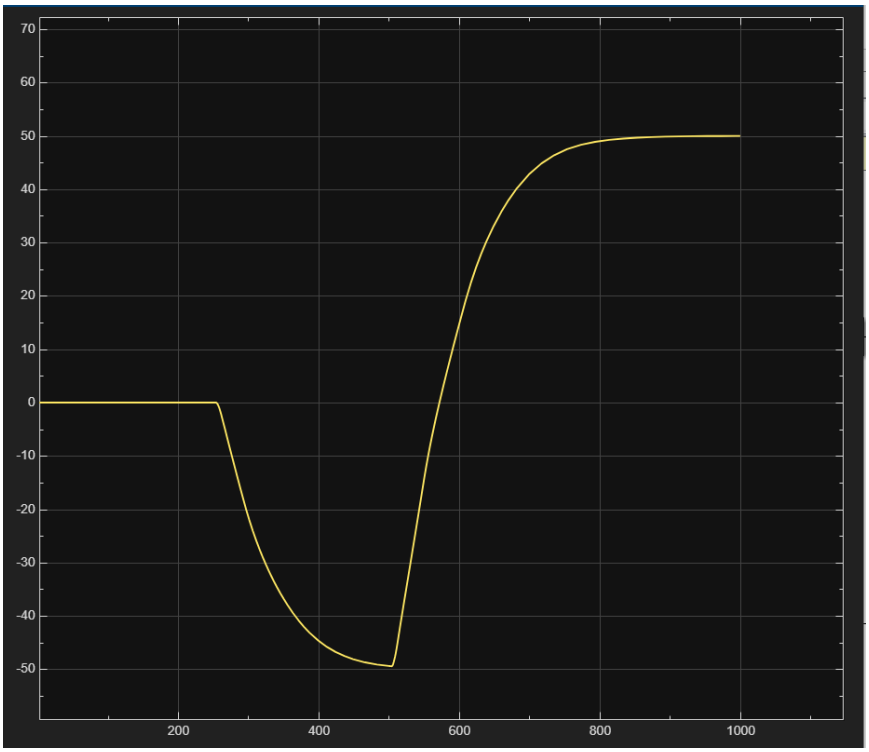


Figure 13 – Control signal

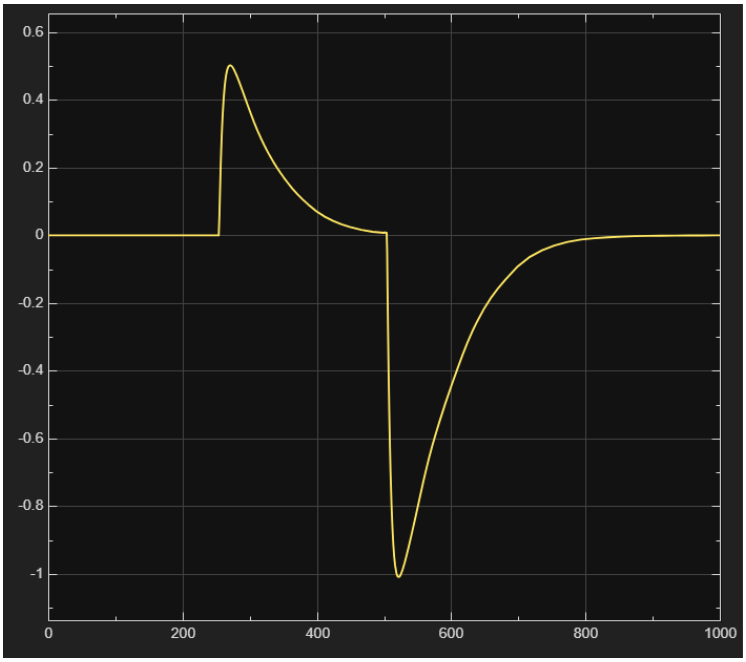


Figure 14 – Output signal

Discussion – system with disturbance:

Control signal

The control signal is initially zero, which is correct, since at the beginning there is neither a disturbance nor an error. When the disturbance appears, the controller reacts by generating a negative control signal that decreases to about −50. After the disturbance changes to its minimum value, the control signal rises to about +50. In both cases the steady-state control signal equals −d, i.e. the controller fully compensates the disturbance.

Plant output signal

The system output is initially zero. After the positive disturbance acts, the output reaches a maximum deviation of about 0.5 and then gradually decreases towards 0. When the disturbance switches to its minimum value, the output deviates in the opposite direction and reaches approximately −1, after which it returns to the reference value again. The second deviation is about twice as large as the first because the disturbance changes by 100 (from +50 to −50) instead of by 50. Despite the disturbance, the system manages to stabilize and reacts successfully.

Signals at the input of the fuzzy inference system

When the positive disturbance appears, the output becomes positive, so the error becomes negative and reaches approximately −0.5. Then, as the controller compensates the disturbance, the error gradually returns to zero.

When the disturbance changes to its minimum value, the output deviates in the negative direction, so the error becomes positive and reaches approximately +1, after which it decreases towards zero again. This value is outside the defined range of E ([-0.8, 0.8]), so the fuzzy inference system treats it as the boundary value (out-of-range inputs are clipped) and the controller acts with the maximum strength defined for a large positive error; if needed, the range of E could be extended to [-1, 1]. The signal of the change of error shows pronounced pulses exactly at the moments of abrupt disturbance changes, while in the other intervals it stays close to zero.

Conclusion:

The system successfully rejects the disturbance at the plant input.